# Clase 19 · Administración de memoria, recolección de basura y pozos de memoria

Fundamentos de Sistemas Embebidos · M.C. José Mauricio Matamoros de María y Campos · miércoles 4 nov 2026 · Tema 3 · Apuntes y resumen de estudio

## PARTE 1 · Apuntes completos

*Comunicación entre procesos y RPC, por qué paginar la memoria (ejercicio de la tabla), el administrador de asignación y la fragmentación, los cinco algoritmos de asignación, recolección de basura y memory pools para sistemas embebidos. Continúa la Clase 18.*

**Fuente**

NotebookLM (audio FSE-Clase19.m4a). Lo que no cuadra con la referencia técnica se marca con ⚠️.

### 1. Comunicación entre procesos y RPC

- **Memoria compartida:** una región accesible por varios procesos. **Paso de mensajes:** datos encapsulados por un canal o protocolo. **RPC:** pedir que se ejecute una función en otro proceso o sistema y esperar la respuesta.
- En un sistema **homogéneo** compilado como una unidad (C o C++) gana la memoria compartida: solo se pasan **apuntadores** («lo más pequeño que pueden mover»), sin serialización ni encriptado.
- En un sistema **distribuido o heterogéneo** (satélites, routers, rovers, nube) se desconoce el lenguaje del otro lado y se usa una *lingua franca* con paso de mensajes (JSON, XML) y su costo.
- Analogías: mensajes = cartas o SMS; memoria compartida = pizarrón común; RPC = WhatsApp con palomitas y respuesta.

Error común: serializar mensajes pesados en un microcontrolador pequeño.

### 2. Por qué paginar: el ejercicio de la tabla

- Antes (Intel 8086 y 80186) la memoria era un bloque continuo sin protección; desde el 286 el hardware detecta violaciones de segmento.
- Memoria de **64 KB** con palabras de 16 bits = **32K localidades**.
- Administrar dirección por dirección: cada renglón lleva dirección (16 bits) + banderas (2 bits: libre/ocupado y limpio/sucio) + PID (8 bits, hasta 256 procesos) = 26 bits, que se redondean a **32 bits (4 bytes)**.
- Tabla = 32K renglones × 4 bytes = **128 KB**.

**Examen**

«Para administrar 64 KB necesito 128 KB: necesito más memoria para administrarla que la memoria que tengo.» Por eso se divide la memoria en **páginas**.

- Con páginas, la tabla cae a decenas o cientos de bytes: NotebookLM dio 64 bytes (páginas de 1 KB) y 128 bytes (páginas de 512 bytes), una fracción que no llena ni una página.
- ⚠️ Las cuentas de bits que dio NotebookLM para las páginas no cuadran entre sí (por ejemplo, 64 KB / 1 KB son 64 páginas, no 32); lo que sí se sostiene es la conclusión de que la tabla se reduce drásticamente.

Error común: llevar el control byte por byte o palabra por palabra.

## 3. El administrador de asignación (MAS) y la fragmentación

- Funciones: cargar y descargar procesos, reservar y liberar bloques, rastrear dónde está la información (RAM, GPU o swap) y reducir la fragmentación.
- Un disco magnético es hasta 1000 veces más lento que la RAM (un SSD, ~100 veces); abusar de la swap causa *lag* o colapso.
- **Fragmentación:** la memoria libre queda en huecos no contiguos; aunque la suma alcance, ningún hueco basta para un proceso nuevo.
- En sistemas embebidos sin memoria virtual, la fragmentación severa impide cargar la aplicación.

Error común: confundir memoria libre total con memoria **contigua** disponible.

## 4. Los 5 algoritmos de asignación

**Examen**

El profesor dijo: «Esto nos lleva a los cinco algoritmos que vienen en el examen.»

| Algoritmo | Cómo busca | Ventaja / desventaja |
|---|---|---|
| First Fit | Desde el inicio, el primer hueco donde quepa | Fragmenta el inicio y reinicia el barrido cada vez |
| Next Fit | Recuerda la última posición y sigue hacia adelante | Suele ser el más rápido; fragmentación dispersa |
| Best Fit | Revisa toda la tabla y toma el hueco más ajustado | Lento; deja residuos minúsculos inutilizables |
| Worst Fit | Revisa toda la tabla y toma el hueco más grande | Útil si el proceso va a crecer (filosofía de Chrome) |
| Quick Fit | Consulta una estructura auxiliar de huecos clasificados por tamaño | Muy veloz; gasta memoria extra |

- ⚠️ NotebookLM dijo que Quick Fit usa una tabla hash con búsqueda binaria en O(log N); una tabla hash sería O(1) y el árbol ordenado O(log N), así que mezcló las dos.

Error común: creer que Best Fit siempre es el mejor; puede fragmentar peor que Worst Fit a largo plazo.

## 5. Recolección de basura

- Mecanismo que rastrea y libera memoria de objetos que ya no son alcanzables; nació con lenguajes orientados a objetos como Java.
- **Contador de referencias:** +1 al asignar el objeto a una variable, −1 al salir de su ámbito (la llave de cierre `}`); en cero se marca como basura.
- **Mark & Sweep:** marca y borra los registros en cero; deja la memoria fragmentada. ⚠️ NotebookLM lo llamó «no bloqueante»; en general los recolectores clásicos sí detienen el programa.
- **Mark & Compact:** además mueve los objetos vivos a un extremo contiguo; **bloquea** el sistema mientras copia, y con muchos objetos de vida corta pasa más tiempo compactando que calculando (Linux tuvo que reescribirlo en sus inicios por su efecto en los pipes).
- **Generacional:** supone que la mayoría de los objetos mueren jóvenes; la generación joven se limpia seguido con Mark & Sweep sin compactar y los sobrevivientes pasan a la vieja, que se revisa poco y se compacta de forma esporádica.
- ⚠️ El contador de referencias y Mark & Sweep son estrategias distintas; NotebookLM las mezcló al explicar el barrido.
- Por eso C y C++ dominan en tiempo real embebido: sin recolector, no hay pausas impredecibles.

Error común: creer que el recolector sobrescribe con ceros; solo libera el registro de la tabla.

## 6. Pozos de memoria (memory pools)

- Reserva estática y determinista de bloques fijos en **potencias de 2** (32, 64, 128 bytes…), sin usar el heap dinámico. Pensada para sistemas que operan 10 o 20 años (collar GPS de una mascota, un satélite).
- Al compilar se analiza el árbol de llamadas y se calcula el peor caso.
- **Jerarquía de asignación:** pozo del tamaño exacto → siguiente pozo mayor → heap general → si no hay espacio contiguo, suspender el proceso, copiarlo a una región mayor, duplicar los pozos y reanudar.
- El ingeniero ajusta a mano el **linker script** tras 3 a 5 pruebas de compilación.
- Ejemplo: se piden 95 bytes y se asignan 128 (la potencia de 2 siguiente); sobran 33 bytes por velocidad y cero fragmentación.

Error común: confiar en que el compilador fija bien los límites sin ajustar el linker.

## Contexto — no examen

- La película *Coco* para explicar el contador de referencias (la foto en el altar).
- El chiste de TCP con el saludo de tres vías.
- Un chip GPS en un gato que debe durar 20 años sin abrirse.
- Consejos de vida y bromas sobre relaciones y profesiones.
- Comentarios sobre noticias, universidades y política (no se resumen aquí).

## Pendiente

- Siguiente tema anunciado: memoria virtual.
- Estudiar los cinco algoritmos de asignación, confirmados para el examen.

## PARTE 2 · Resumen para estudiar

*Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.*

| Tema | Idea para recordar |
|---|---|
| Memoria compartida | Se pasan apuntadores; sin costo en sistemas homogéneos |
| Mensajes y RPC | Para sistemas distribuidos o heterogéneos; cuestan serialización |
| Tabla por dirección | 64 KB necesitan 128 KB de tabla: por eso se pagina |
| Fragmentación | Hay memoria libre pero no contigua |
| First Fit | Primer hueco desde el inicio |
| Next Fit | Sigue desde la última posición |
| Best Fit | El hueco más ajustado (lento, residuos) |
| Worst Fit | El hueco más grande (por si crece) |
| Quick Fit | Listas por tamaño (rápido, gasta memoria) |
| Mark & Sweep / Compact / Generacional | Barrer / mover (bloquea) / por edades |
| Memory pools | Bloques en potencias de 2; determinista; sin fragmentación |

**Hilo conductor**

Administrar memoria es repartir bloques sin que se fragmente: se pagina para que la tabla no pese más que la memoria, se elige un algoritmo de asignación, y en tiempo real embebido se evita el heap y la recolección de basura con pozos fijos.